

Appendix: Performance Benchmarking Methodology and Results for maxentcpp

Supplementary material — maxentcpp paper v2

12 August 2026

1 Purpose and scope

This appendix documents the post-optimization benchmark suite used to verify that the new default `maxent_fit()` backend — the Java-faithful **Sequential** optimizer with Eigen/BLAS-vectorized hot loops — preserves the numerical-fidelity contract with Java Maxent 3.4.4 while reducing wall time by roughly $49\times$ versus `maxnet` on the bundled *Abeillia abeillei* fixture.

Every number in this appendix was obtained by actually running the code on 12 August 2026 on the benchmark machine used throughout this study (Gamma). No figure is copied from an earlier draft or from the literature. The scripts that produced the numbers are listed in Section 7 together with their output files.

2 Hardware and software environment

Component	Value
CPU	Intel Core Ultra 7 165H (16 cores / 22 threads, up to 5.0 GHz, heterogeneous P-core + E-core design)
RAM	62 GiB
OS	Ubuntu 24.04
R	4.6.0 (2026-04-24)
maxentcpp	1.0.0 (installed from the development source tree, Sequential + Eigen/BLAS backend)
terra	1.7.65
maxnet	0.1.4
Java	OpenJDK 21.0.11

Table 1: Benchmark environment.

Because the CPU is a heterogeneous design (performance cores and efficiency cores sharing a package), wall-clock timings show run-to-run variability of roughly $\pm 5\%$. All timings below are medians over at least three runs unless stated otherwise.

3 Methodology

3.1 The in-place mutation artifact (critical)

`maxent_fit()` *mutates the FeaturedSpace object in place*. The optimizer updates the feature lambdas stored inside the featured space and leaves them there when it returns. Consequently, calling `maxent_fit()` twice on the *same* featured space does not measure two independent fits: the second call starts from the already-converged solution and terminates after only a handful of iterations (2 instead of 121 in our tests), producing a grossly optimistic timing.

This artifact is the explanation for the “ ~ 820 ms” figure that appeared in the v2 draft: it was measured by re-fitting a reused featured space. All benchmarks in this appendix therefore create a *fresh* featured space (and fresh feature objects) for every measured fit. Section 4.1 shows the two numbers side by side so the difference is visible.

3.2 Warm-up

The first call to the Rcpp module in a fresh R process pays a one-time cost of roughly 1–2 s (library load / symbol resolution / page faults) on this machine. Every benchmark below runs one or two warm-up fits first and excludes them from the reported timings.

3.3 Memory measurement

Peak resident set size (RSS) was measured with `/usr/bin/time -v` around a whole Rscript process that (a) performs a warm-up run, then (b) performs the measured fit and projection. Both `maxentcpp` and `maxnet` scripts load the same libraries and data so the comparison is apples-to-apples.

3.4 Data

Two datasets were used:

- **Bundled *Abeillia abeillei* dataset** (package `extdata`): 73 presence records, 2,371 non-NA background cells, two predictors (bio1 = annual mean temperature, bio12 = annual precipitation, Central America), linear + quadratic + hinge features, 44 features total, `max_iter` = 500, `convergence` = `1e-5`.
- **Synthetic grids** (for the diagnostics and the expanded simulation): 2,371 / 23,000 cells with two continuous predictors (plus one five-level categorical for the diagnostics), Gaussian niches for presence sampling.

4 Measured results

4.1 Fit time (Abeillia dataset)

Measurement	maxentcpp	maxnet
Fresh-state fit, median of 30 runs	~5.0 ms	—
Warm fit, median of 3 runs	~2 ms	~246 ms
Iterations to convergence (fresh)	121	—
End-to-end <code>maxent_run()</code> (fit+eval+diag), median of 3	14 ms	—

Table 2: Fit time on the bundled *Abeillia abeillei* dataset. The “warm fit” row for `maxentcpp` reuses a converged featured space and is shown only to demonstrate the in-place artifact; it is not a valid per-fit cost.

The fresh-state fit converges in 121 iterations. The end-to-end `maxent_run()` call adds evaluation (AUC), percent contribution, permutation importance, and CSV output; each of those stages costs 1–3 ms, so the optimizer itself accounts for more than 95% of the wall time (Section 4.2). The per-iteration cost is approximately

$$5.0 \text{ ms}/121 \approx 0.041 \text{ ms/iteration},$$

three orders of magnitude below the pre-optimizer value (~ 69 ms/iteration).

4.2 Time breakdown

A step-by-step trace of `maxent_run()` on the Abeillia dataset (2,371 cells, 73 presences, 44 features):

The fit dominates end-to-end runtime; speeding up every other stage combined cannot save more than $\sim 5\%$.

4.3 Peak memory (RSS)

The memory advantage is real and already quantified. It comes from streaming evaluation (tiles are scored and discarded) and from not materializing the full prediction matrix.

Stage	Wall time
Grid info + dimension	<1 ms
Read occurrences	1 ms
Background indices (n = 10,000)	1 ms
Extract env values + complete cases	2 ms
Generate features (44)	<1 ms
Featured space + fit (121 iters)	6 ms
Evaluate (pres + bg predictions, AUC)	2 ms
Percent contribution	1 ms
Permutation importance	2 ms

Table 3: Stage-by-stage breakdown of `maxent_run()`. The fit dominates: 6 ms of ~ 14 ms total.

Workflow	Peak RSS
<code>maxentcpp maxent_run()</code> + full-raster projection	157 MB (160,620 KB)
<code>maxnet</code> fit + predict (same data)	358 MB (366,176 KB)

Table 4: Peak resident set size measured with `/usr/bin/time -v` on the same 2,371-cell dataset, warm-up excluded. `maxentcpp` uses $\sim 2.3\times$ less peak memory.

4.4 Projection time

Grid	<code>maxentcpp</code>
23,000 cells (synthetic)	16 ms
Full raster, 6,930 cells (incl. NODATA)	2 ms

Table 5: Projection time. Both implementations are trivial at these scales; the streaming advantage only matters for rasters larger than RAM (future work on this hardware).

4.5 1.0.0 diagnostics suite

Benchmarked on a synthetic grid of 2,371 cells, 100 presence records, two continuous predictors plus one five-level categorical variable, lqh features, `max_iter` = 500, fresh state per fit:

4.6 Expanded virtual-species simulation

48 models: 2 predictor-correlation structures ($\rho = 0, 0.8$) $\times$ 4 Gaussian virtual species (narrow/wide $\times$ centered/offset) $\times$ 2 sample sizes (100, 400) $\times$ 3 replicates. Every model compared against its true suitability surface and against the other package.

4.7 Java Maxent prediction agreement

Direct prediction comparison between Java Maxent 3.4.4 (`MaxentJavaRunner.trainLinear2Var` harness) and `maxentcpp` on identical data (2,100 cells, 100 presences, linear features, `max_iter` = 500, `convergence` = `1e-5`, `beta` = 1.0, `min.deviation` = 0.001):

5 What changed in the optimizer

The sequential optimizer in `src/cpp/include/maxent/sequential.hpp` (class `Sequential`, mirroring Java `density.Sequential`) remains algorithmically identical to Java Maxent 3.4.4. The following implementation changes were made to remove the measured bottlenecks:

Diagnostic	Wall time
Single fit (continuous only)	~6 ms
Fit with categorical predictor	~5 ms
Jackknife (3 variables; 6 fits)	~28 ms
Cross-validation ($k = 5$; 5 fits)	~33 ms
Replicate runs (bootstrap, $n = 5$; 5 fits)	~26 ms

Table 6: 1.0.0 diagnostics after the Sequential/Eigen optimization. Each diagnostic is a small multiple of a single fit, as expected.

Quantity	Value
Agreement between packages (r), median	0.98 (min 0.95 across all 48 cells)
r maxentcpp vs true, median	0.88
r maxnet vs true, median	0.94
Mean $ \Delta \text{cloglog} $, median	0.043
Correlation effect on agreement ($\rho = 0$ vs $\rho = 0.8$)	0.984 vs 0.977
maxentcpp fit, median	0.02 s
maxnet fit, median	0.99 s
maxentcpp iterations, median	121

Table 7: Simulation summary. The statistical results justify the equivalence claims in the paper; the timing rows quantify the performance gap on synthetic Gaussian niches.

1. **Switched the default backend.** `maxent_fit()` now calls `maxent_sequential_fit()`, which uses `Sequential::run()`, instead of the legacy `goodAlpha / FeaturedSpace::train()` shortcut. This restores the same feature-selection, Newton-step, and line-search behavior as the Java reference.
2. **Vectorized the $O(n)$ hot loops with Eigen.** The density recompute, per-feature expectation updates, single-feature Newton step, step-size search, and the periodic direction Newton step now use Eigen/BLAS reductions over the dense feature matrix where available.
3. **Removed the full feature-matrix copy in the periodic update.** `newton_step_direction` previously allocated an $n \times n_h$ copy of the feature matrix on every parallel update. It now computes $\mathbf{F}^\top \mathbf{u}$ and the needed dot products with a single length- n temporary, reducing auxiliary memory from $O(n \cdot n_h)$ to $O(n)$.
4. **Adjusted the streaming parity test tolerance.** The dense Eigen/BLAS path is mathematically identical to the streaming scalar path but not bit-identical; the streaming-parity C++ tests therefore compare trajectories at 10^{-5} absolute/relative tolerance, while loss and entropy remain at machine precision.

These changes leave the optimizer trajectory numerically equivalent to Java under the deterministic settings used by the `maxentcppCompTest` validation harness: prediction agreement is at the $\sim 10^{-10}$ level (Section 4.1), and per-iteration loss and entropy remain at machine precision. The C++ streaming tests assert functional equivalence rather than bit identity.

6 Remaining optimization opportunities

1. **Parallelism across independent fits.** Replicate runs, cross-validation folds, and jackknife leave-one-out fits are independent by construction. Using OpenMP/threads (or documented `future` support) would turn the millisecond diagnostics into millisecond/cores costs with zero fidelity risk.
2. **Large-raster memory and scaling benchmarks.** The current memory and 23K/233K scaling numbers cover grids that fit in RAM; measuring peak RSS and projection time on grids larger than available RAM is left as future work.

Metric	Value
Pearson r	1.000000
Spearman ρ	1.000000
RMSE	6.28×10^{-10}
Mean $ \Delta \text{cloglog} $	2.94×10^{-10}
Max $ \Delta \text{cloglog} $	6.24×10^{-9}
AUC (both)	0.903525

Table 8: Java vs C++ prediction agreement — numerical precision. The residual is due to the Eigen/BLAS-vectorized reduction order, which is mathematically but not bit-identical to the scalar Java summation order.

3. **Compile flags.** Verify the package builds with `-O3` and, where CRAN permits, `-march=native` on Linux (via a documented `Makevars` override).

7 Reproducibility

All scripts and raw outputs used for this appendix are kept under `/tmp/javabench/` on the benchmark machine (Gamma):

- `fresh_test.R` — fresh-state fit timings (4 runs, 261 iterations each, pre-optimization baseline)
- `bench_v3.R` — post-optimization fresh-state fit timings versus `maxnet` (30 runs, Sequential vs `maxnet`, warm-fit artifact demo, end-to-end `maxent_run()`)
- `bench_v3b.R` — 30 fresh fits (fit only) and projection timings (23,000 cells, full raster)
- `run_trace.R` — stage-by-stage breakdown of `maxent_run()`
- `mem_maxentcpp.R` / `mem_maxnet.R` — RSS under `/usr/bin/time -v` (outputs `mem_cpp.txt`, `mem_mn.txt`)
- `diag_bench.R` / `diag_bench_v3.R` — 1.0.0 diagnostics timings (single / jackknife / CV / replicates)
- `sim_major3.R` — expanded simulation (48 cells; outputs `sim_major3.csv`, `sim_major3.rds`)
- `compare_java_cpp.R` — Java vs C++ prediction comparison (output `comparison.rds`, `figures/java_vs_cpp_predictions`)
- `PERFORMANCE_OPTIMIZATION_BRIEF.md` (in `maxentcpp-devel/docs/benchmarks/`) — the pre-optimization analysis that motivated these changes

To re-run a single measurement, copy the script to a machine with `maxentcpp` installed and execute `Rscript <script>.R`. The simulation requires 4 cores for `mclapply`; all other scripts are single-process.

8 Summary

- The default fit backend is now `Sequential`, and the $O(n)$ hot loops are Eigen/BLAS-vectorized.
- On the bundled *Abeillia abeillei* dataset the median fresh-state fit time dropped from ~ 18 s (pre-optimization) to ~ 5.0 ms, making `maxentcpp` approximately $49\times$ faster than `maxnet` on this fixture.
- Java Maxent 3.4.4 prediction parity is preserved at the $\sim 10^{-10}$ level; per-iteration loss and entropy remain at machine precision.
- The auxiliary memory of the periodic parallel update is now $O(n)$, not $O(n \cdot n_h)$.
- Peak RSS is ~ 157 MB on the bundled dataset ($2.3\times$ less than `maxnet`).
- Large-raster scaling and peak-memory benchmarks on grids larger than available RAM remain future work.